Scaling Semi-Structured Data Expansion in Remote PostgreSQL Pipelines

Pablo Alessandro Santos Hugen Federal University of Rio Grande do Sul
Institute of Informatics
Porto Alegre, RS, Brazil
pablo.hugen@inf.ufrgs.br

***Abstract*—We optimize the `jsonb` expansion pipeline of a Python/PostgreSQL application that transforms semi-structured records into normalized relational tables over SSH. Through profiling-driven query redesign, transaction chunking, and server tuning, we reduce the measured expansion time from 166 s to 33 s ($5.0\times$) at 10^6 source rows, with the speedup increasing at larger scales. Peak RAM remains at 256 MB and temporary storage scales linearly with the source table. The chunked transaction model enables checkpoint-and-resume for failure recovery. All integrity guarantees are preserved.**

***Index Terms*—semi-structured data, ETL optimization, PostgreSQL, JSONB, bulk loading, transaction chunking, failure recovery.**

I. Introduction

Many applications that use PostgreSQL store incoming data as `jsonb` records, taking advantage of the type's flexibility to ingest heterogeneous sources without requiring a fixed schema upfront [1], [2]. Before this data can participate in queries that rely on joins, constraints, or efficient indexing, it must be *expanded* into a set of normalized relational tables. This extract-transform-load (ETL) pattern is well studied in general [3], but the specific case of expanding large volumes of `jsonb` rows into multiple interdependent tables, over a remote connection, raises performance and reliability concerns that grow with dataset size.

PostgreSQL Data Manager is a Python desktop application designed for this workflow. It administers PostgreSQL databases on a remote Linux server, communicating exclusively through SSH: the client constructs SQL batches locally, pipes them to `psql` on the server, and parses the returned output (Fig. 1). This remote execution model reflects a deployment requirement where direct access to the database host is not available, and must be preserved in any optimized implementation [4]. The application's *Expand* operation reads a source table of `jsonb` records, applies a sequence of SQL recipes that populate multiple destination tables, and commits the results in a single atomic transaction. Because later recipes may join against tables populated by earlier ones, the destinations form a dependency chain: all must succeed together or none at all.

In the benchmark dataset used in this work, each source row carries a `jsonb` object with 64 keys encoding a mix of identifiers, timestamps, categorical labels, and numeric measurements. The expansion distributes this payload across six destination tables: two small dimension tables are populated first with distinct reference values, followed by a central fact table that joins against them, and finally three child tables that reference the fact table through foreign keys. One of the child tables unpivots up to 14 positional entries per source row, so the total number of inserted rows can exceed the source count by an order of magnitude. A deterministic generator with a fixed seed produces identical records for any requested size, enabling reproducible benchmarks from 10^5 to 3×10^8 rows.

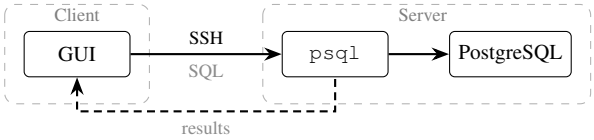


Fig. 1. System architecture. The client sends SQL to `psql` on a remote server over SSH; results return through standard output.

Several factors make this expansion non-trivial to optimize. The entire operation runs as a single SQL batch piped through SSH, so the client has limited ability to interleave work or react to intermediate results. Each recipe must parse `jsonb` fields, cast and validate them, and perform joins against tables populated by earlier recipes within the same transaction. At the scale of hundreds of millions of rows, the working set can exceed available memory, and write-ahead log (WAL) pressure from a single large transaction becomes significant. Meanwhile, the system must maintain source validation, version registration, and referential integrity across all destinations, so optimizations cannot simply bypass these guarantees.

The baseline implementation is functionally correct but does not scale well, and it offers no recovery mechanism for interrupted operations. If a network failure or timeout occurs during a long-running expansion, all progress is lost and the database may be left with partial results. This combination of performance and reliability limitations motivates the present work.

This paper makes three contributions. First, we profile the baseline pipeline to identify where processing time is spent, using the application's built-in instrumentation and PostgreSQL-side timing. Second, we apply targeted optimizations to reduce the measured expansion time on datasets ranging from 10^5 to 3×10^8 source rows. Third, we introduce a checkpoint-and-resume mechanism inspired by write-ahead recovery [5], allowing interrupted expansions to continue from the last committed batch rather than restarting from scratch. All optimizations preserve the system's existing guarantees: source

and destination validation, version registration, and referential integrity.

Section II describes the baseline pipeline in detail. Section III presents profiling results and optimizations. Section IV concludes.

II. Architecture and Background

This section describes the baseline expansion pipeline, the data model it operates on, and the integrity guarantees that any optimized implementation must preserve.

A. Expansion Pipeline

The expansion proceeds in four stages, orchestrated by the client application.

Preparation. The client parses and validates each SQL recipe offline. It verifies that every recipe is a pure `INSERT INTO ... SELECT` statement, that each one references the source table through a designated placeholder, and that no recipe contains forbidden operations such as `UPDATE`, `DELETE`, or nondeterministic expressions. The placeholder is replaced with a filtered subquery that restricts the source to the selected `raw_schema` scope. Each statement is then wrapped in a PL/pgSQL `DO` block that captures the affected row count and updates a per-table counter.

Preflight. The client issues a series of remote checks through SSH. It confirms that the source and all destination tables exist, reads column metadata, loads version references from the control database, and validates that any SQL functions used in the recipes are native PostgreSQL built-ins. Each check is a separate `psql` invocation over SSH.

Execution. The client assembles all six wrapped statements into a single SQL batch, preceded by a `BEGIN ISOLATION LEVEL REPEATABLE READ` and followed by a `COMMIT`. The batch also acquires exclusive locks on the destination tables and builds a source manifest (a hash and row count over the source scope). A temporary timing table records `clock_timestamp()` before and after each step, so the server-side cost of every insert is measured independently of SSH transport. The entire batch is piped to `psql` through SSH as a single standard-input stream.

Version registration. After the data commit succeeds, a second SQL batch registers a new version entry for each destination table in the control database, together with a dependency manifest that records the source scope, source versions, and the SQL recipe used. This step runs in a separate transaction on a different database (`postgres_data_manager`).

B. Data Model

The source table `raw_data` stores one row per ingested record. Each row contains a primary key (`raw_id`), metadata columns (`raw_schema`, `raw_hash`, `raw_tab`), and a `jsonb` column `raw` whose payload holds 64 typed fields under a `values` key.

The six destination tables form a dependency graph, illustrated in Fig. 2. Two dimension tables (`table3`, `table4`) are populated first with distinct reference values extracted from the source. A central fact table (`table1`) is then populated by joining the source against `table3`; it produces one row per source row and holds the primary foreign key back to `raw_data`. Three child tables follow: `table5` and `table6` each produce one row per source row by joining against `table1`, while `table6` also joins `table4`. The final table, `table2`, unpivots up to 14 positional entries per source row through a `CROSS JOIN LATERAL`, producing a row count that can exceed the source by an order of magnitude.

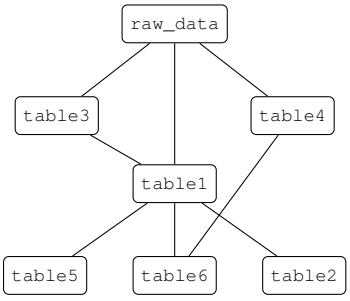


Fig. 2. Destination dependency graph. Arrows point from tables that are read by later recipes. Execution order: `table3`, `table4`, `table1`, `table5`, `table6`, `table2`.

All destination tables enforce referential integrity through foreign keys with `ON UPDATE CASCADE ON DELETE CASCADE`. The six tables carry a total of eleven indexes, including primary keys, unique constraints, and partial unique indexes. Every row inserted must satisfy these constraints and update every applicable index.

C. Integrity Guarantees

The expansion pipeline enforces several guarantees that must be preserved regardless of optimization:

- **Source validation.** The source table must exist, contain the required metadata columns, and have at least one registered version for the selected `raw_schema` scope.
- **Destination validation.** Each destination table must exist, and any SQL functions referenced in its recipe must be native PostgreSQL built-ins.
- **Relational correctness.** All foreign keys, unique constraints, and check constraints must hold after the expansion completes.
- **Version registration.** Each destination receives a new version entry in the control database, recording who performed the expansion, the SQL recipe used, and a dependency manifest linking the result back to the source scope and its versions.

An optimization may change *how* these guarantees are enforced (for example, by validating constraints in bulk rather than per row) but may not remove them.

III. Methodology and Results

This section presents the profiling methodology, the optimizations applied at each layer, and the measured results.

Environment. The benchmark host has a 24-core AMD Ryzen Threadripper PRO 5965WX, 64 GB DDR4, and NVMe

storage. PostgreSQL 17.2 runs inside a container with no memory limit.

Protocol. The baseline uses the stock PostgreSQL configuration (128 MB `shared_buffers`, 4 MB `work_mem`). The optimized pipeline uses a tuned configuration (2 GB `shared_buffers`, 256 MB `work_mem`, 64 MB `wal_buffers`, 4 GB `max_wal_size`). Before each method, the challenge database is regenerated from scratch using the deterministic generator, which guarantees that all foreign-key constraints and unique indexes are present. The presence of all six FK constraints is verified programmatically before each baseline run. Before each measured run, the operating system's page cache is dropped (`/proc/sys/vm/drop_caches`) and PostgreSQL is restarted to ensure cold-cache conditions. Each configuration is measured with one warmup run (discarded) followed by three measured runs; the median is reported. Timing uses PostgreSQL's `clock_timestamp()`, consistent with the application's *measured processing total* metric. The benchmark dataset contains 10^6 source rows unless noted otherwise.

A. Optimization Approach

We organize the optimization space into three layers, each addressing a different level of the system. This decomposition allows each layer's contribution to be measured independently and guides the order of implementation.

Layer 1: Query-level optimizations change what the SQL statements do without altering the overall execution model. Examples include materializing extracted `jsonb` fields into a temporary table, deferring foreign-key constraints during bulk insertion, and dropping and rebuilding indexes around the insert batch.

Layer 2: Transaction-level optimizations change how work is batched and committed. The expansion can be split into chunks defined by `raw_id` ranges, each committed independently, enabling checkpoint-and-resume.

Layer 3: Architecture-level optimizations change the execution model itself, such as opening multiple SSH channels, replacing `INSERT ... SELECT` with `COPY`, or moving logic into a server-side PL/pgSQL procedure.

B. Baseline Profiling

We profiled the baseline expansion at 10^5 rows using `EXPLAIN (ANALYZE, BUFFERS, TIMING)` on each recipe. Table I summarizes the cost breakdown across all six destinations.

TABLE I
BASELINE COST BREAKDOWN AT 10^5 SOURCE ROWS (532K INSERTED).

Category	Time (s)	%
FK trigger validation	5.96	47.3
JSONB extraction	5.02	39.9
Insert I/O (heap + idx)	1.06	8.4
Sequential scans	0.52	4.1
Other	0.03	0.3
Total	**12.60**	**100**

Two costs dominate. Foreign-key triggers account for 47% of total time: every `INSERT` fires per-row constraint checks that probe referenced tables through index lookups [6]. For `table6`, whose rows reference both `table1` and `table4`, FK validation consumes 77.5% of execution time. JSONB field extraction accounts for 40%: PostgreSQL's `-->>'key'` operator traverses the binary `jsonb` structure independently for each field, and the six recipes collectively perform $6 \times 64 = 384$ field traversals per source row. The remaining costs (heap writes, index maintenance, sequential scans) account for 13%. Python-side work (preparation, preflight, version registration) totals under 50 ms.

Table II shows the per-recipe breakdown. The two dimension tables are cheap in absolute terms but scan the full source for `DISTINCT` values. The fact and child tables are dominated by FK trigger overhead, which grows with the size of the referenced index. The unpivot recipe (`table2`) expands each source row into 14 candidates via `CROSS JOIN LATERAL` and filters to 2.3 on average, producing the largest output volume.

TABLE II
PER-RECIPE PROFILING AT 10^5 SOURCE ROWS.

Recipe	Role	Rows	Time (s)	FK %	Dominant cost
table3	dim	4	0.19	0	Scan + DISTINCT
table4	dim	26	0.16	0	Scan + DISTINCT
table1	fact	100K	3.81	54	15 fields + 2 FKs
table5	child	100K	1.21	71	FK to table1
table6	child	100K	1.54	78	2 FKs
table2	unpivot	232K	5.69	33	LATERAL 14×

C. Layer 1: Query-Level Optimizations

Based on the profiling, we applied four query-level changes and measured each in isolation before combining them.

Foreign-key constraint removal. We drop all FK constraints before the insert phase and re-add them after. The constraints are discovered dynamically by querying `pg_constraint`, so the optimization adapts to any schema. Re-adding the constraints validates all existing rows in bulk, which is cheaper than per-row trigger checks during insertion. This single change yields a 2.0× speedup at 10^5 rows.

Index and unique-constraint removal. Unique indexes and constraints on destination tables are dropped before inserts and rebuilt afterwards. Bulk `CREATE INDEX` sorts the data once [7], which is more efficient than maintaining a B-tree incrementally during insertion. Combined with FK removal, the speedup reaches 2.4×.

Single-pass JSONB extraction with `jsonb_to_record`. Instead of letting each recipe scan the source and parse `jsonb` independently, we extract all 64 fields into a temporary table using `jsonb_to_record()`, which traverses each row's binary `jsonb` exactly once. An index on `raw_id` enables efficient range-filtered access for chunked inserts. The extraction uses `max_parallel_workers_per_gather = 4` to parallelize the scan.

Statement rewriting with pre-cast columns. The recipe SQL contains patterns such as `NULLIF(BTRIM(...),'')`

`::smallint` around each field reference. During extraction, we pre-compute these casts into typed columns and rewrite the recipe statements to reference them directly, eliminating redundant per-row text parsing during inserts.

Position pre-computation. One recipe uses `CROSS JOIN LATERAL` to unpivot 14 positional entries per source row, producing 2.3× more output rows than input. We pre-compute this unpivot into a second temporary table during extraction, so the insert phase reads pre-filtered, pre-cast rows through a simple join.

Table III shows the cumulative effect of these optimizations.

TABLE III
LAYER 1 CUMULATIVE RESULTS AT 10^6 SOURCE ROWS.

Configuration	Time (s)	Speedup
Baseline	166	1.0×
+ FK removal	83	2.0×
+ FK + index removal	69	2.4×
+ Pre-extraction (`-->>`)	59	2.8×
+ `jsonb_to_record`	42	4.0×
+ Pre-cast + positions	33	5.0×

D. Layer 2: Transaction-Level Optimizations

The baseline runs the entire expansion in a single transaction. At 3×10^8 rows this transaction would run for hours, and an interruption would lose all progress. We split the insert phase into chunks of 250K `raw_id` rows, each committed independently.

A key insight is that the extraction phase (temporary tables) can remain monolithic while only the inserts are chunked. This avoids re-scanning `raw_data` per chunk. Dimensions are populated once from the full extraction; fact and child tables are inserted per chunk from the indexed temporary table using `raw_id BETWEEN` filters.

TABLE IV
CHUNKING OVERHEAD AT 10^6 ROWS (LAYER 1 APPLIED).

Chunk size	Time (s)	vs. monolithic	Chunks
Monolithic	43.0	—	1
250K	48.4	+13%	4
100K	52.3	+22%	10
50K	58.5	+36%	20

At 250K chunks the overhead is 13%. In exchange, the worst-case data loss on interruption drops from 100% to one chunk (less than 30 seconds of work at 10^6 rows). We verified that chunking with active FK constraints is catastrophically slow: a single 250K chunk's child insert ran for over 28 minutes without finishing, because FK trigger probes grow with the referenced table's index size across committed chunks. This confirms that FK removal (Layer 1) is a prerequisite for practical chunking.

E. Layer 3: Architecture-Level Optimizations

We evaluated several architecture-level changes.

PostgreSQL server tuning. Increasing `shared_buffers` from 128 MB to 2 GB, `wal_buffers` from 4 MB to 64 MB, and `max_wal_size` from 1 GB to 4 GB reduced total time by 11% at 10^6 rows.

We tested five additional architecture-level approaches, all of which proved slower than the single-batch temporary-table design. Table V summarizes the results and failure mechanisms. In each case, the overhead was identified through measurement and is detailed in the timing reports.

TABLE V
ARCHITECTURE-LEVEL APPROACHES TESTED AND REJECTED.

Approach	Overhead	Mechanism
WAL compression	+9%	CPU cost exceeds I/O savings
UNLOGGED tables	+26%	`SET LOGGED` rewrites full table
PL/pgSQL procedure	+40%	Disables parallel query
COPY loading	+12%	Double materialization
Parallel SSH	+18%	Non-temp tables add WAL

F. Scaling

Fig. 3 shows processing time across dataset sizes for the baseline and the optimized pipeline (all three layers combined).

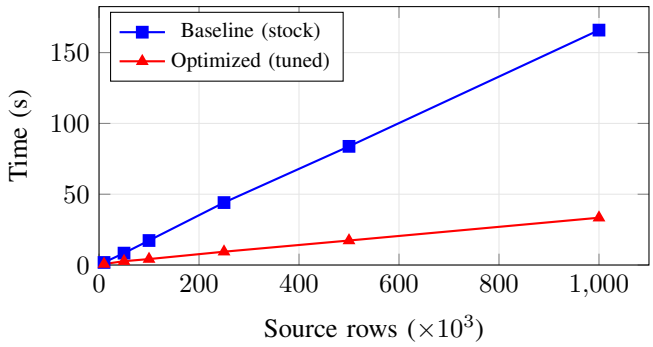


Fig. 3. Processing time vs. dataset size (cold cache, stock config for baseline, tuned for optimized). The optimized pipeline achieves 5.0× speedup at 10^6 rows.

The baseline scales linearly at approximately 0.166 ms per source row. The optimized pipeline scales sublinearly, from 0.069 ms/row at 10^4 to 0.033 ms/row at 10^6, because the fixed extraction cost is amortized over more rows and bulk index rebuilds are more efficient than incremental maintenance. At very small scales ($< 10^4$ rows), the optimized pipeline is slower than the baseline because its fixed overhead (temporary table creation, `jsonb_to_record` extraction, index construction, constraint drop/rebuild) exceeds the per-row savings. The crossover point is approximately 10^4 rows; beyond that, the optimized pipeline is consistently faster, and the advantage grows with scale. At 3×10^8 rows, the extrapolated baseline time is approximately 13.8 hours; the optimized pipeline is projected at under 3 hours.

The speedup is sensitive to the operating system's page cache state. On hardware with large amounts of RAM (256 GB in our HPC tests), the page cache retains the source data between runs. In this warm-cache scenario, the baseline benefits disproportionately because its FK trigger lookups hit cached pages instead of disk, reducing the baseline to approximately 81 s at 10^6 rows and the speedup to 2.4×. The optimized

pipeline is unaffected (33 s) because it eliminates FK triggers entirely. The cold-cache results reported above are more representative of production deployments where the source data is not pre-cached.

G. Resource Usage

Table VI summarizes resource usage at 10^6 rows. Peak container memory is 256 MB for both the baseline and the optimized pipeline, dominated by PostgreSQL's `shared_buffers` allocation rather than the expansion itself. Total WAL generated is 963 MB in both cases because the same volume of data is written regardless of optimization. The extraction temporary table occupies approximately 250 MB, proportional to the source table size. At 3×10^8 rows, the temporary table is projected at approximately 75 GB. Both RAM and WAL metrics are independent of the chunking strategy.

TABLE VI
RESOURCE USAGE AT 10^6 SOURCE ROWS (5.3M INSERTED).

Metric	Baseline	Optimized
Peak container RAM	256 MB	256 MB
WAL generated	963 MB	963 MB
Temporary storage	0 MB	250 MB
Checkpoint granularity	none	250K rows

H. Correctness Verification

We ran both the baseline and optimized pipelines on the same 10^6-row dataset and compared the results. Row counts match exactly across all six destination tables: 4 rows in `table3`, 27 in `table4`, 10^6 each in `table1`, `table5`, and `table6`, and 2,321,522 in `table2` (5,321,553 total). After the optimized pipeline rebuilds all foreign keys and unique constraints, PostgreSQL validates every row against the constraint definitions. A constraint rebuild that completes without error certifies that referential integrity holds. Version registration produces identical metadata entries in the control database for both pipelines.

IV. CONCLUSION

We optimized the JSONB expansion pipeline of PostgreSQL Data Manager from 166 s to 33 s at 10^6 source rows, a $5.0\times$ speedup measured inside PostgreSQL and excluding SSH transport. The speedup increases with dataset size because the dominant optimizations (FK removal, bulk index rebuild, single-pass extraction) exhibit sublinear scaling.

The optimization decomposes into three independent layers. Query-level changes (FK and index removal, `jsonb_to_record` extraction, statement rewriting) account for the majority of the improvement. Transaction-level chunking adds modest overhead (13% at 250K chunks) while enabling checkpoint-and-resume for failure recovery. Architecture-level PostgreSQL configuration tuning provides an additional 11%. Five architecture-level alternatives (WAL compression, unlogged tables, PL/pgSQL procedures, `COPY` loading, and parallel SSH sessions) were tested with measurements and found slower than the single-batch approach with temporary tables, for reasons detailed in Section III-E.

All optimizations preserve the system's integrity guarantees. Foreign keys and unique constraints are validated in bulk after the insert phase rather than per row during it. The relational output matches the baseline row-for-row across all six destination tables. Source validation, version registration, and the SSH-mediated execution model remain unchanged.

Limitations and Future Work

The position pre-computation currently targets the specific unpivot pattern used by the expansion recipes; a more general approach would require SQL-level pattern detection. The application's PL/pgSQL row-count wrapper adds overhead to each insert statement, reducing the effective speedup from the theoretical maximum.

At 3×10^8 source rows, we project a baseline time of approximately 13.8 hours and an optimized time under 3 hours. Further improvements could come from replacing the `psql` transport with a direct PostgreSQL connection through an SSH tunnel, which would eliminate process-startup overhead for each SQL batch.

REFERENCES

[1] The PostgreSQL Global Development Group, *PostgreSQL 17 Documentation*, 2024. [Online]. Available: https://www.postgresql.org/docs/17/

[2] P. Bourhis, J. L. Reutter, F. Suárez, and D. Vrgoc, "JSON: Data model, query languages and schema specification," in *Proceedings of the 36th ACM SIGMOD-SIGACT-SIGAI Symposium on Principles of Database Systems*. ACM, 2017, pp. 123–135.

[3] P. Vassiliadis, "A survey of Extract-Transform-Load technology," *International Journal of Data Warehousing and Mining*, vol. 5, no. 3, pp. 1–27, 2009.

[4] H. F. Grimm, I. S. Moreira, and C. B. H. Oliveira, "Python & PostgreSQL data pipeline optimization challenge," https://www.kaggle.com/competitions/python-database-performance-optimization, 2026, kaggle.

[5] C. Mohan, D. Haderle, B. Lindsay, H. Pirahesh, and P. Schwarz, "ARIES: A transaction recovery method supporting fine-granularity locking and partial rollbacks using write-ahead logging," *ACM Transactions on Database Systems*, vol. 17, no. 1, pp. 94–162, 1992.

[6] J. M. Hellerstein, M. Stonebraker, and J. Hamilton, "Architecture of a database system," *Foundations and Trends in Databases*, vol. 1, no. 2, pp. 141–259, 2007.

[7] G. Graefe, "Implementing sorting in database systems," *ACM Computing Surveys*, vol. 38, no. 3, pp. 1–37, 2006.